scikit-learn

Data Preprocessing

OVERVIEW

What scikit-learn gives you

1. **Preprocessing**
   split, impute, scale, encode, ...
2. **Supervised learning**
   regression, classification, ...
3. **Unsupervised learning**
   clustering, dimension reduction, ...
4. **Model evaluation**
   metrics on held-out data
5. **Model selection**
   tune hyperparameters on validation

Key Objects

```
BaseEstimator
├── TransformerMixin     ← Data Transformer (preprocessing)
├── ClassifierMixin      ← Estimator (classification)
├── RegressorMixin       ← Estimator (regression)
├── ClusterMixin, etc.   ← Estimator (clustering)
└── ...
# Transformers  : .fit() / .transform()
# Estimators    : .fit() / .predict()
```

DATA SPLIT

Split before you work with data

We assume that some data cleaning procedures that do not depend on **data values** have already been applied. (Examples?)

Goal: find a model that performs well on **unseen data.**

- Split into train / validation / test sets.
- Fit the model on train, select on validation.
- After the split, never use test-set information.

Typical proportions — tune to your dataset size.

Code example

`scikit-learn` supports **random** split.
How you split the dataset should depend on the dataset and the problem you're solving.

```
from sklearn.model_selection import train_test_split

# supervised
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=87)

# unsupervised (no labels)
X_tr, X_te = train_test_split(X, test_size=0.2, random_state=87)
```

How to use the Transformer objects?

Any transformation that learns from data is fit only on the training set,

then applied to train and test.

TRANSFORMATION

Which one is wrong?

```
X = pd.DataFrame(...)
X_train, X_test = train_test_split(X, test_size = 0.2, random_state=87)
```

(1)

```
tf = Transformer(...)
tf.fit(X_train)
X_tr = tf.transform(X_train)
X_te = tf.transform(X_test)
```

(2)

```
tf = Transformer(...)
tf.fit(X)
X_tr = tf.transform(X_train)
X_te = tf.transform(X_test)
```

TRANSFORMATION

Which one is wrong?

(1) fit on train

```
tf = Transformer(...)
tf.fit(X_train)
X_tr = tf.transform(X_train)
X_te = tf.transform(X_test)
```

OK — Params learned from train only. Test stays unseen.

(2) fit on all of X

```
tf = Transformer(...)
tf.fit(X)
X_tr = tf.transform(X_train)
X_te = tf.transform(X_test)
```

X — fit(X) sees the test set.

Code template

```
tf = Transformer(...)
tf.fit(X_train)
X_tr_t = tf.transform(X_train)
X_te_t = tf.transform(X_test)

# shortcut: fit + transform in one call
tf = Transformer(...)
X_tr_t = tf.fit_transform(X_train)
X_te_t = tf.transform(X_test)
```

TRANSFORMATION

Common transformers

Transformer	What it does	Use case
`SimpleImputer`	Fill missing values (mean / median / mode)	Missing data
`StandardScaler`	Zero mean, unit variance	Scaling numeric features
`MinMaxScaler`	Rescale to a range (default 0–1)	Bounded models
`OrdinalEncoder`	Categories -> ordered integers	Ordinal categories
`OneHotEncoder`	Categories -> binary columns	Nominal categories

Missing value imputation

```
from sklearn.preprocessing import SimpleImputer

scaler = SimpleImputer(strategy = 'mean')
X_tr = scaler.fit_transform(X_train)
X_te = scaler.transform(X_test)
```

Possible strategies: **`mean`**, **`median`**, **`constant`**, **`most_frequent`**

Scaling for quantitative variables

Models that rely on **distances** or **gradient-based optimization** need features on a comparable scale.

`LogisticRegression`	optimizer assumes similar scales
`Ridge / Lasso / ElasticNet`	penalty is sensitive to magnitude
`SVM`	margins & kernels are distance-based
`KNeighbors`	dominant features distort distance
`KMeans`	Euclidean distance skews clusters
`PCA`	variance-based; big scales dominate

Code examples

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_tr = scaler.fit_transform(X_train)
X_te = scaler.transform(X_test)

from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
X_tr = scaler.fit_transform(X_train)
X_te = scaler.transform(X_test)
```

Encoding for categorical variables

Any model needing numeric input requires categorical variables to be encoded first.

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(handle_unknown="ignore")
encoder.fit(X_train)
X_tr = encoder.transform(X_train)
X_te = encoder.transform(X_test)
```

TRANSFORMATION - Dimensionality reduction

(Optional) Dimensionality reduction

Unsupervised, but useful in preprocessing — to reduce noise, remove redundancy, simplify the feature space. Fit on train, apply to test.

The rule: a method is safe for preprocessing only if it supports **out-of-sample transform** — i.e. it has a **`.transform()`** method.

PCA

✓ has .transform()

Linear projection onto top variance directions.

Isomap

✓ has .transform()

Nonlinear manifold; geodesic distances + MDS.

t-SNE

✕ no .transform()

Transductive — fit_transform only, viz only.

Code examples

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
pca.fit(X_train)
X_tr = pca.transform(X_train)
X_te = pca.transform(X_test)
```

Save the results

1 Processed data

via `to_csv()` -> `data/processed/`

```
X_train_processed
X_test_processed
y_train
y_test
```

2 Fitted transformers

via `pickle` -> `model/`

```
imputer
scaler
encoder
pca
```

Multiple models?

If they need different preprocessing, save each set separately under `data/processed/ModelName/` and `model/ModelName/`.

Modification

Also, you may modify the directory structure as you want!

Some notes...

- Many steps can be done by pandas — new features, grouping categories, handling outliers, domain transforms.
- You can do preprocessing at any point, as long as they are applied consistently to train and test.
- ***Golden rule: fit on train · apply to both · never leak the test set.***